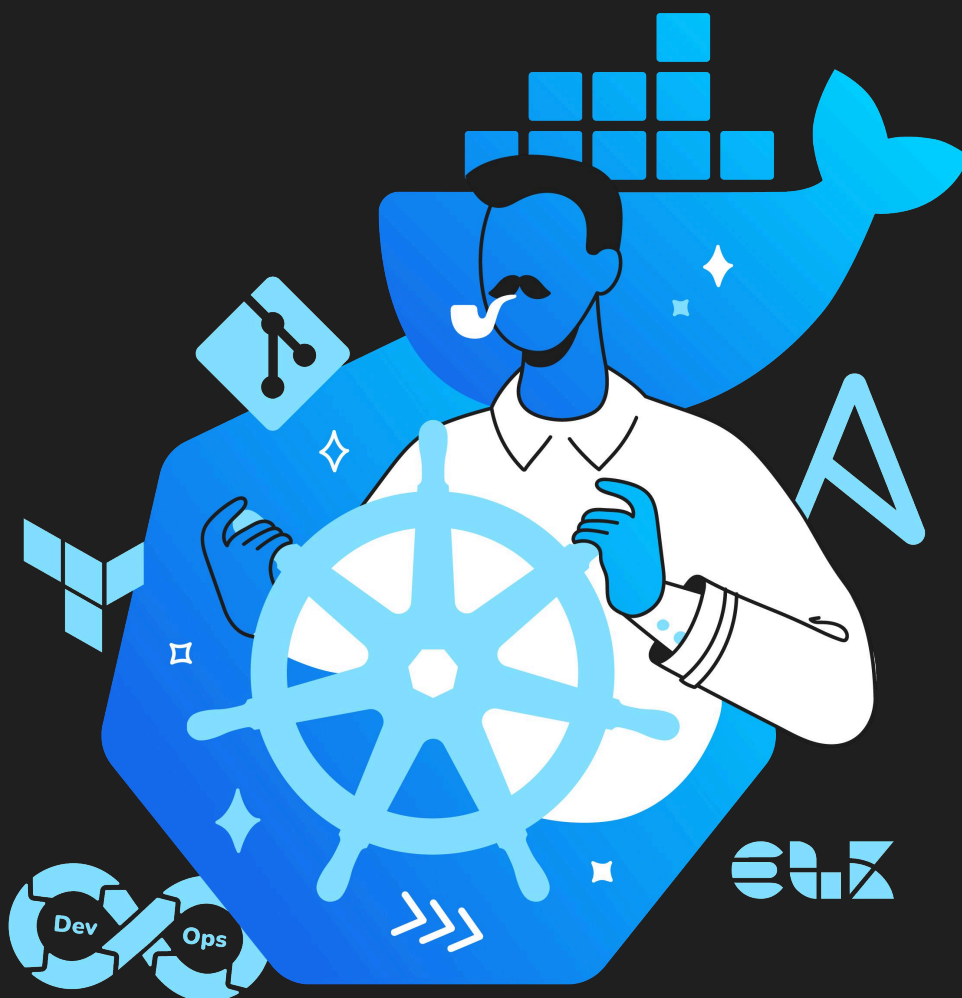


Методический материал

РАБОТА С ДЕПЛОЙМЕНТАМИ, РЕВИЗИЯМИ, ОТКАТАМИ И СКАЛИРОВАНИЕМ ПРИЛОЖЕНИЙ В K8S

Урок 10.





Перед началом работ убедитесь, что у вас установлены следующие компоненты:

- Docker.
- Kubectl.
- K3d.

С установкой Docker необходимой версии различными способами с учетом добавления альтернативных репозиторий и зеркал можно ознакомиться в методических материалах предыдущих уроков.

Установка Kubectl и K3d приведена ниже:

#01 УСТАНОВКА KUBECTL ОПРЕДЕЛЕННОЙ ВЕРСИИ:

Зададим желаемую версию (в нашем случае это 1.30):

```
$ VERSION=v1.30.0
```

```
filipp@user-IdeaPad-3-15IAU7:~$ VERSION=v1.30.0
```

Произведем скачивание указанной версии Kubectl:

```
$ curl -LO  
"https://cdn.dl.k8s.io/release/${VERSION}/bin/linux/amd64 /kubectl"
```

```
filipp@user-IdeaPad-3-15IAU7:~$ curl -LO "https://cdn.dl.k8s.io/release/${VERSION}/bin/linux/  
amd64/kubectl"  
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current  
           Dload  Upload   Total   Spent    Left   Speed  
100 49.0M  100 49.0M    0     0  16.0M      0  0:00:03  0:00:03 --:--:-- 16.0M
```



Добавим права на выполнение для Kubectl:

```
$ chmod +x kubectl
```

```
filipp@user-IdeaPad-3-15IAU7:~$ chmod +x kubectl
```

Переместим в директорию Bin:

```
$ sudo mv kubectl /usr/local/bin/
```

```
filipp@user-IdeaPad-3-15IAU7:~$ sudo mv kubectl /usr/local/bin/
```

Проверим корректность установки выбранной нами версии:

```
kubectl: $ kubectl version --client
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl version --client
Client Version: v1.30.0
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
```

Утилита Kubectl успешно установлена.



#02 УСТАНОВКА K3D ПОСЛЕДНЕЙ СТАБИЛЬНОЙ ВЕРСИИ:

```
$ curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install .sh |  
bash
```

```
filipp@user-IdeaPad-3-15IAU7:~$ curl -s https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash  
Preparing to install k3d into /usr/local/bin  
k3d installed into /usr/local/bin/k3d  
Run 'k3d --help' to see what you can do with it.
```

Проверка успешности установки:

```
$ k3d version
```

```
filipp@user-IdeaPad-3-15IAU7:~$ k3d version  
k3d version v5.8.3  
k3s version v1.31.5-k3s1 (default)
```

k3d успешно установлен.

Переходим к работе с деплойментами.



#03 ПРАКТИЧЕСКАЯ РАБОТА ПО СОЗДАНИЮ ДЕПЛОЙМЕНТА:

Для начала запустим кластер K3d с именем "Mycluster":

```
$ k3d cluster create mycluster
```

```
filipp@user-IdeaPad-3-15IAU7:~$ k3d cluster create mycluster
INFO[0000] Prep: Network
INFO[0000] Created network 'k3d-mycluster'
INFO[0000] Created image volume k3d-mycluster-images
INFO[0000] Starting new tools node...
INFO[0001] Creating node 'k3d-mycluster-server-0'
INFO[0001] Pulling image 'ghcr.io/k3d-io/k3d-tools:5.8.3'
INFO[0003] Starting node 'k3d-mycluster-tools'
INFO[0004] Pulling image 'docker.io/rancher/k3s:v1.31.5-k3s1'
INFO[0012] Creating LoadBalancer 'k3d-mycluster-serverlb'
INFO[0013] Pulling image 'ghcr.io/k3d-io/k3d-proxy:5.8.3'
INFO[0017] Using the k3d-tools node to gather environment information
INFO[0017] HostIP: using network gateway 172.19.0.1 address
INFO[0017] Starting cluster 'mycluster'
INFO[0017] Starting servers...
INFO[0017] Starting node 'k3d-mycluster-server-0'
INFO[0020] All agents already running.
INFO[0020] Starting helpers...
INFO[0020] Starting node 'k3d-mycluster-serverlb'
INFO[0026] Injecting records for hostAliases (incl. host.k3d.internal) and for 2 network
o CoreDNS configmap...
INFO[0028] Cluster 'mycluster' created successfully!
INFO[0028] You can now use it like this:
kubectl cluster-info
```



Убедимся, что кластер установлен успешно:

```
$ kubectl cluster-info
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl cluster-info
Kubernetes control plane is running at https://0.0.0.0:33641
CoreDNS is running at https://0.0.0.0:33641/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
Metrics-server is running at https://0.0.0.0:33641/api/v1/namespaces/kube-system/services/https:metrics-server:https/proxy
To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

Кластер создан успешно, теперь создадим неймспейс "Mobsf" для работы с будущим деплойментом:

```
$ kubectl create ns mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl create ns mobsf
namespace/mobsf created
```

Создадим деплоймент "Mobsf" в ранее подготовленном одноименном неймспейсе на основе Docker-образа Mobsf версии v3.1.1:

```
$ kubectl create deployment mobsf
--image=opensecurity/mobile-security-framework
mobsf:v3.1.1 -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl create deployment mobsf --image=opensecurity/mobile-security-framework-mobsf:v3.1.1 -n mobsf
deployment.apps/mobsf created
```



Создадим службу для деплоймента Mobsf в том же неймспейсе:

```
$ kubectl expose deployment mobsf --port=8000 -- type=NodePort -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl expose deployment mobsf --port=8000 --type=NodePort -n mobsf
service/mobsf exposed
filipp@user-IdeaPad-3-15IAU7:~$
```

Для доступа к веб-интерфейсу установленного сервиса выполним команду для отображения EXTERNAL-IP нашего системного ингресс-контроллера в k3d по умолчанию (службы с типом Loadbalancer):

```
$ kubectl get svc -n kube-system
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl get svc -n kube-system
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kube-dns	ClusterIP	10.43.0.10	<none>	53/UDP,53/TCP,9153/TCP	2m58s
metrics-server	ClusterIP	10.43.233.154	<none>	443/TCP	2m58s
traefik	LoadBalancer	10.43.140.45	172.19.0.2	80:31104/TCP,443:31468/TCP	2m39s

Выполним команду просмотра службы Mobsf в одноименном неймспейсе и запомним выделенный данной службе порт (порт может принимать значение из определенного диапазона — пятизначный порт, начинающийся с цифры "3"):

```
$ kubectl get svc -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl get svc -n mobsf
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
mobsf	NodePort	10.43.181.104	<none>	8000:31776/TCP	113s



Убедимся, что Pod в неймспейсе Mobsf запущен и находится в статусе

```
"running": $ kubectl get pods -n mobsf
```

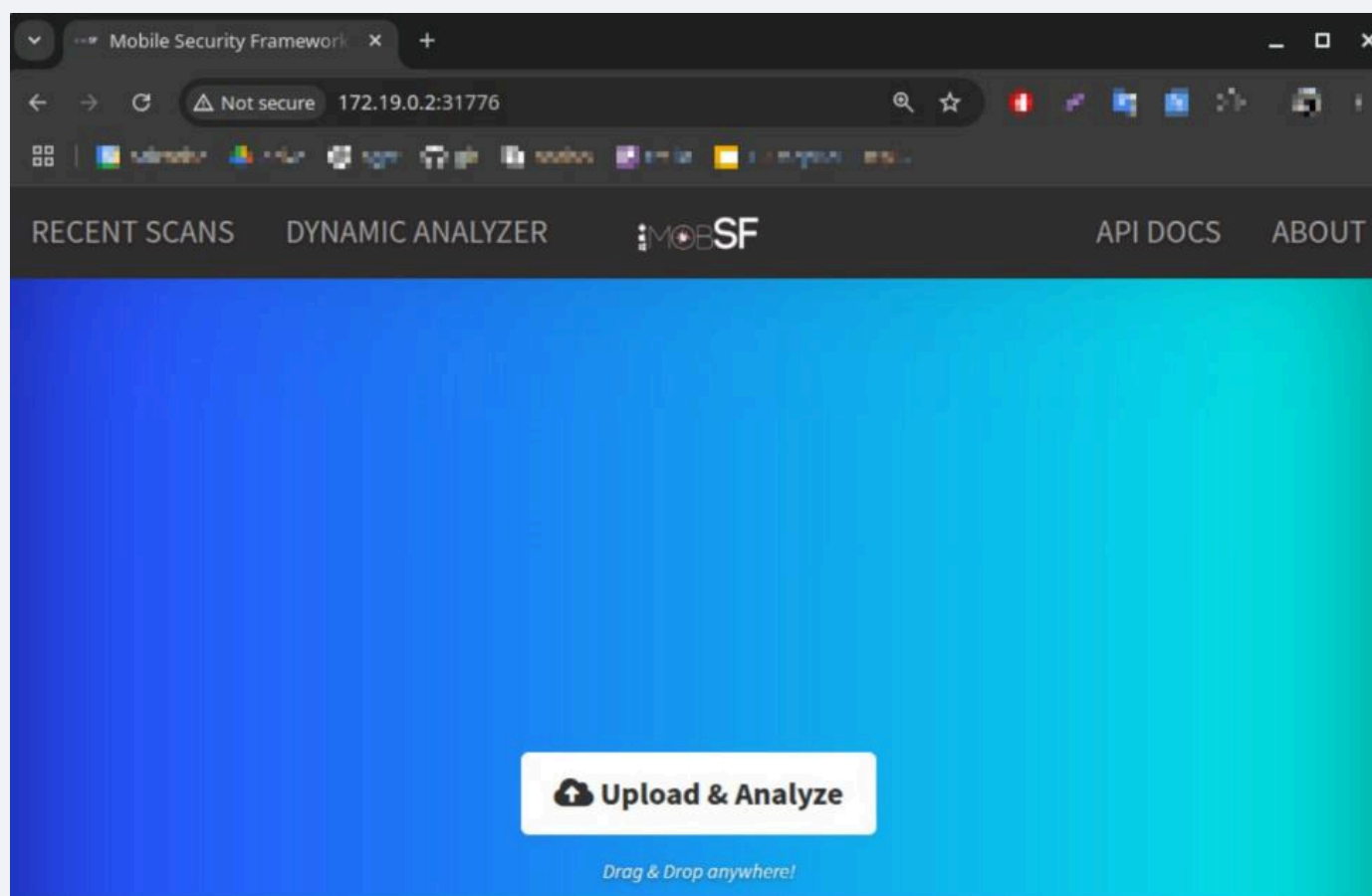
```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl get pods -n mobsf
NAME                                READY   STATUS    RESTARTS   AGE
mobsf-858667b46b-8zmb6             1/1     Running   0           2m30s
```



External-ip службы Traefik и порт службы Mobsf (не забудьте подставить именно ваш IP адрес и порт службы, руководствуясь выводами предыдущих выполненных команд):

В адресной строке браузера:

```
http://<external-ip>:<mobsf-service port>/
```





Веб-интерфейс установленного с помощью Deployment приложения в K3d кластере запущен и работает. Теперь перейдем к практической работе по обновлению, откату и репликации деплоймента.

Вызовем ручной режим модификации деплоймента:

```
$ kubectl edit deployment mobsf -n mobsf
```

```
spec:
  progressDeadlineSeconds: 600
  replicas: 1
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: mobsf
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
  template:
    metadata:
      creationTimestamp: null
    labels:
      app: mobsf
```



Команда успешно применена.

Проинспектируем сервис Redis в Docker swarm в Pretty-формате вывода:

```
$ docker service inspect --pretty redis
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service inspect --pretty redis
ID:                zgir14ih2ztzyumifqr28czjj
Name:              redis
Service Mode:      Replicated
  Replicas:        2
Placement:
UpdateConfig:
  Parallelism:      1
  Delay:            10s
  On failure:       pause
  Monitoring Period: 5s
  Max failure ratio: 0
  Update order:     stop-first
RollbackConfig:
  Parallelism:      1
  On failure:       pause
  Monitoring Period: 5s
  Max failure ratio: 0
  Rollback order:   stop-first
ContainerSpec:
  Image:            redis:7@sha256:fbdbaea47b9ae4ecc2082ecdb4e1cea81e32176ff
d9
  Init:             false
Resources:
Endpoint Mode:     vip
```



В разделе "Спец" отредактируем значение количества Relicas с 1 до 2:

```
spec:
  progressDeadlineSeconds: 600
  replicas: 2 ←
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: mobsf
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
  template:
    metadata:
      creationTimestamp: null
    labels:
      app: mobsf
```

Сохраняем изменения:

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl edit deployment mobsf -n mobsf
deployment.apps/mobsf edited
```



Выполним команду для просмотра количества Pods в неймспейсе Mobsf:

```
$ kubectl get pods -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl get pods -n mobsf
NAME                                READY   STATUS    RESTARTS   AGE
mobsf-858667b46b-7msvw             1/1     Running   0           29s
mobsf-858667b46b-8zmb6             1/1     Running   0           9m35s
```

Количество репликаций изменено в соответствии с указанным ранее количеством.

Теперь произведем попытку обновления деплоймента Mobsf, указав несуществующую версию Docker-image:

```
$ kubectl set image deployments/mobsf mobile-security- framework-
mobsf=opensecurity/mobile-security-framework- mobsf:v3.1.2 -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl set image deployments/mobsf mobile-security-framework-mobsf=
opensecurity/mobile-security-framework-mobsf:v3.1.2 -n mobsf
deployment.apps/mobsf image updated
```



Выполним команду просмотра статуса текущего обновления деплоймента:

```
$ kubectl rollout status deployment/mobsf -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl rollout status deployment/mobsf -n mobsf
Waiting for deployment "mobsf" rollout to finish: 1 out of 2 new replicas have been updated...
```

Выполним команду просмотра текущих Pods в неймспейсе Mobsf:

```
$ kubectl get pods -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl get pods -n mobsf
NAME                                READY   STATUS              RESTARTS   AGE
mobsf-675c486b6f-lg62t             0/1     ImagePullBackOff    0           71s
mobsf-858667b46b-7msvw             1/1     Running             0           2m41s
mobsf-858667b46b-8zmb6             1/1     Running             0           11m
```

Обратите внимание на идентификаторы Pods: 2 Pod в статусе Running и 1 Pod в статусе ErrImagePull.

Выполним команду просмотра текущих Replicasets (rs) в неймспейсе Mosf:

```
$ kubectl get rs -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl get rs -n mobsf
NAME                                DESIRED   CURRENT   READY   AGE
mobsf-675c486b6f                   1         1         0       90s
mobsf-858667b46b                   2         2         2       12m
```



Идентификатор Replicaset с Desired и Current равными 2 соответствуют двум Pods в статусе Running, а идентификатор Replicaset с Desired и Current равными 1 соответствуют одному Pod в статусе ErrImagePull:

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl get pods -n mobsf
NAME                                READY    STATUS      RESTARTS   AGE
mobsf-675c486b6f-1g02t             0/1      ErrImagePull 0           2m1s
mobsf-858667b46b-7msvw             1/1      Running      0           3m31s
mobsf-858667b46b-8zmb6             1/1      Running      0           12m
filipp@user-IdeaPad-3-15IAU7:~$ kubectl get rs -n mobsf
NAME                                DESIRED   CURRENT   READY   AGE
mobsf-675c486b6f                    1         1         0       2m3s
mobsf-858667b46b                    2         2         2       12m
```

Выполним Rollback (Rollout undo) текущего обновления деплоймента в неймспейсе Mobsf:

```
$ kubectl rollout undo deployment/mobsf -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl rollout undo deployment/mobsf -n mobsf
deployment.apps/mobsf rolled back
```

Выполним команду просмотра текущих Pods в неймспейсе Mobsf:

```
$ kubectl get pods -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl get pods -n mobsf
NAME                                READY    STATUS      RESTARTS   AGE
mobsf-858667b46b-7msvw             1/1      Running      0           6m45s
mobsf-858667b46b-8zmb6             1/1      Running      0           15m
```



Pod в статусе ErrImagePull больше не присутствует в выводе команды.
На данный момент запущено 2 Pods со статусом Running (рабочей версии v3.1.1).

Выполним еще одну попытку обновления деплоймента
в неймспейсе Mobsf, на этот раз обновим версию Docker-image
на существующую v3.5.0:

```
$ kubectl set image deployments/mobsf mobile-security- framework-  
mobsf=opensecurity/mobile-security-framework- mobsf:v3.5.0 -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl set image deployments/mobsf mobile-security-framework-mobsf=  
opensecurity/mobile-security-framework-mobsf:v3.5.0 -n mobsf  
deployment.apps/mobsf image updated
```

Выполним команду просмотра текущих Pods в неймспейсе Mobsf
с флагом "-w" для просмотра всей цепочки обновления Pods:

```
$ kubectl get pods -n mobsf -w
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl get pods -n mobsf -w  
NAME                READY   STATUS             RESTARTS   AGE  
mobsf-79db4f6cd7-fsg45 0/1     ContainerCreating   0           24s  
mobsf-858667b46b-7msvw 1/1     Running             0           7m51s  
mobsf-858667b46b-8zmb6 1/1     Running             0           16m
```



Обратите внимание на саму процедуру последовательного обновления:

Сперва добавляется и стартует новый pod с версией v3.5.0 (его идентификатор отличается от идентификатора pods с версией v3.1.1)

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl get pods -n mobsf -w
```

NAME	READY	STATUS	RESTARTS	AGE
mobsf-79db4f6cd7-fsg45	0/1	ContainerCreating	0	24s
mobsf-858667b46b-7msvw	1/1	Running	0	7m51s
mobsf-858667b46b-8zmb6	1/1	Running	0	16m
mobsf-79db4f6cd7-fsg45	1/1	Running	0	74s
mobsf-858667b46b-7msvw	1/1	Terminating	0	8m41s
mobsf-79db4f6cd7-fhbb6	0/1	Pending	0	0s
mobsf-79db4f6cd7-fhbb6	0/1	Pending	0	0s
mobsf-79db4f6cd7-fhbb6	0/1	ContainerCreating	0	0s
mobsf-79db4f6cd7-fhbb6	1/1	Running	0	1s
mobsf-858667b46b-8zmb6	1/1	Terminating	0	17m

Затем последовательно терминируется 1 из 2 Pods с версией v3.1.1, только затем стартует и запускается новый Pod с версией v3.5.0 (его идентификатор частично совпадает с уже работающим Pod версии v3.5.0), после этого терминируется оставшийся 2 из 2 Pod с версией v3.1.1).

Выполним команду для просмотра истории обновлений деплоймента (Revisions):

```
$ kubectl rollout history deployment/mobsf -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl rollout history deployment/mobsf -n mobsf
deployment.apps/mobsf
REVISION  CHANGE-CAUSE
2          <none>
3          <none>
4          <none>
```



Выполним команды для детального просмотра всех имеющихся реверсий:

```
$ kubectl rollout history deployment/mobsf -n mobsf -- revision 2
$ kubectl rollout history deployment/mobsf -n mobsf -- revision 3
$ kubectl rollout history deployment/mobsf -n mobsf -- revision 4
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl rollout history deployment/mobsf -n mobsf --revision 2
deployment.apps/mobsf with revision #2
Pod Template:
  Labels:      app=mobsf
              pod-template-hash=675c486b6f
  Containers:
    mobile-security-framework-mobsf:
      Image:      opensecurity/mobile-security-framework-mobsf:v3.1.2
      Port:       <none>
      Host Port:  <none>
      Environment: <none>
      Mounts:     <none>
  Volumes:      <none>
  Node-Selectors: <none>
  Tolerations:  <none>

filipp@user-IdeaPad-3-15IAU7:~$ kubectl rollout history deployment/mobsf -n mobsf --revision 3
deployment.apps/mobsf with revision #3
Pod Template:
  Labels:      app=mobsf
              pod-template-hash=858667b46b
  Containers:
    mobile-security-framework-mobsf:
      Image:      opensecurity/mobile-security-framework-mobsf:v3.1.1
      Port:       <none>
      Host Port:  <none>
      Environment: <none>
      Mounts:     <none>
  Volumes:      <none>
  Node-Selectors: <none>
  Tolerations:  <none>

filipp@user-IdeaPad-3-15IAU7:~$ kubectl rollout history deployment/mobsf -n mobsf --revision 4
deployment.apps/mobsf with revision #4
Pod Template:
  Labels:      app=mobsf
              pod-template-hash=79db4f6cd7
  Containers:
    mobile-security-framework-mobsf:
      Image:      opensecurity/mobile-security-framework-mobsf:v3.5.0
      Port:       <none>
      Host Port:  <none>
      Environment: <none>
      Mounts:     <none>
  Volumes:      <none>
  Node-Selectors: <none>
  Tolerations:  <none>
```



Ревизия #2 — неуспешная попытка Rolling-update на несуществующую версию v3.1.2

Ревизия #3 — Rollback обратно на версию v3.1.1

Ревизия #4 — успешный Rolling-update до версии v3.5.0

Теперь выполним скалирование репликаций приложения до 3-х, но не в ручном режиме изменения конфигурации деплоймента, а командой:

```
$ kubectl scale deployment/mobsf --replicas=3 -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl scale deployment/mobsf --replicas=3 -n mobsf
deployment.apps/mobsf scaled
```

Выполним команду просмотра текущих Pods в неймспейсе Mobsf:

```
$ kubectl get pods -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl get pods -n mobsf
NAME                                READY   STATUS    RESTARTS   AGE
mobsf-79db4f6cd7-7pckp             1/1     Running   0           20s
mobsf-79db4f6cd7-fhbb6             1/1     Running   0           5m2s
mobsf-79db4f6cd7-fsg45             1/1     Running   0           6m16s
```



Деплоймент успешно проскалирован до 3-х репликаций

Теперь совершим Rollback деплоймента до специфичной Revision (возьмем ревизию #3 - версия приложения v3.1.1):

```
$ kubectl rollout undo deployment/mobsf -n mobsf --to- revision=3
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl rollout undo deployment/mobsf -n mobsf --to-revision=3
deployment.apps/mobsf rolled back
```

Rollback до конкретной версии ревизии успешно выполнен.

Выполним команду просмотра списка текущих ревизий деплоймента Mobsf:

```
$ kubectl rollout history deployment/mobsf -n mobsf
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl rollout history deployment/mobsf -n mobsf
deployment.apps/mobsf
REVISION  CHANGE-CAUSE
2         <none>
4         <none>
5         <none>
```



Нумерация ревизий изменилась, выполним команды детального просмотра всех имеющихся в неймспейсе Mobsf ревизий:

```
$ kubectl rollout history deployment/mobsf -n mobsf -- revision 2
$ kubectl rollout history deployment/mobsf -n mobsf -- revision 4
$ kubectl rollout history deployment/mobsf -n mobsf -- revision 5
```

```
filipp@user-IdeaPad-3-15IAU7:~$ kubectl rollout history deployment/mobsf -n mobsf --revision 2
deployment.apps/mobsf with revision #2
Pod Template:
  Labels:      app=mobsf
              pod-template-hash=675c486b6f
  Containers:
    mobile-security-framework-mobsf:
      Image:      opensecurity/mobile-security-framework-mobsf:v3.1.2
      Port:       <none>
      Host Port:  <none>
      Environment: <none>
      Mounts:     <none>
  Volumes:      <none>
  Node-Selectors: <none>
  Tolerations:  <none>

filipp@user-IdeaPad-3-15IAU7:~$ kubectl rollout history deployment/mobsf -n mobsf --revision 4
deployment.apps/mobsf with revision #4
Pod Template:
  Labels:      app=mobsf
              pod-template-hash=79db4f6cd7
  Containers:
    mobile-security-framework-mobsf:
      Image:      opensecurity/mobile-security-framework-mobsf:v3.5.0
      Port:       <none>
      Host Port:  <none>
      Environment: <none>
      Mounts:     <none>
  Volumes:      <none>
  Node-Selectors: <none>
  Tolerations:  <none>

filipp@user-IdeaPad-3-15IAU7:~$ kubectl rollout history deployment/mobsf -n mobsf --revision 5
deployment.apps/mobsf with revision #5
Pod Template:
  Labels:      app=mobsf
              pod-template-hash=858667b46b
  Containers:
    mobile-security-framework-mobsf:
      Image:      opensecurity/mobile-security-framework-mobsf:v3.1.1
      Port:       <none>
      Host Port:  <none>
      Environment: <none>
      Mounts:     <none>
  Volumes:      <none>
  Node-Selectors: <none>
  Tolerations:  <none>
```



Ревизия #2 — неуспешная попытка Rolling-update на несуществующую версию v3.1.2 (как и было до Rollback на ревизию #3)

Ревизия #4 — успешный Rolling-update до версии v3.5.0 (как и было до Rollback на ревизию #3)

Ревизия #5 — успешный Rollback до версии v3.1.1 (до бывшей ревизии #3 — после успешного Rollback это теперь ревизия #5)

Работа с деплоями, ревизиями, откатами и масштабированием завершена. Текущее состояние кластера пригодится для выполнения методического материала "Методический материал по работе с PV и PVC в K8s" этого урока.

Но при желании, для завершения работ с кластером K3d, выполните команду:

```
$ k3d cluster delete mycluster
```

```
filipp@user-IdeaPad-3-15IAU7:~$ k3d cluster delete mycluster
INFO[0000] Deleting cluster 'mycluster'
INFO[0002] Deleting cluster network 'k3d-mycluster'
INFO[0002] Deleting 1 attached volumes...
INFO[0002] Removing cluster details from default kubeconfig...
INFO[0002] Removing standalone kubeconfig file (if there is one)...
INFO[0002] Successfully deleted cluster mycluster!
```

Кластер K3d успешно удален.